

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AI & ML

LABORATORY OBSERVATION RECORD

LIBRARY – OBJECT ORIENTED PROGRAMMING IN JAVA

1. TITLE OF THE EXPERIMENT

Implementation of Encapsulation, Inheritance, Method Overloading, Method Overriding, Abstraction and Interface in Java

2. AIM

To design and implement a Java program demonstrating important Object-Oriented Programming concepts including encapsulation, inheritance, method overloading, method overriding, abstraction and interfaces.

3. ALGORITHM

Step 1: Start the program.

Step 2: Define the Book class with private data members and a parameterized constructor.

Step 3: Implement getter and setter methods to demonstrate encapsulation and create a display() method to show book details.

Step 4: Define the Person superclass and derive Student and Faculty classes from it to demonstrate inheritance.

Step 5: Use super.display() in the child classes to reuse the parent display() method.

Step 6: Define the Area class with overloaded area() methods for circle and rectangle to demonstrate compile-time polymorphism.

Step 7: Define Vehicle and override display() in Car and Bike to demonstrate runtime polymorphism.

Step 8: Define the abstract Shape class and implement draw() in Circle and Rectangle.

Step 9: Define the Printable interface and implement print() in the Report class.

Step 10: Create objects in main() and invoke the respective methods.

Step 11: Display all results and terminate the program.

4. EXECUTION STEPS

Step 1: Compile the Java source file with the package name task and class name Library.

Step 2: Run the Library program.

Step 3: Observe the BOOK DETAILS section and verify the Book object's details.

Step 4: Observe the INHERITANCE section and verify Student and Faculty details.

Step 5: Observe METHOD OVERLOADING and verify circle and rectangle area calculations.

Step 6: Observe METHOD OVERRIDING and verify the messages from Car and Bike.

Step 7: Observe ABSTRACTION and verify the draw() implementation for Circle and Rectangle.

Step 8: Observe INTERFACE and verify the print() implementation in Report.

Step 9: Compare the displayed output with the expected output.

5. SOURCE CODE

```
package task;
class Book {
    private int bookId;
    private String bookName;
    private String author;
    private double price;
    Book(int id, String name, String auth, double pr) {
        bookId = id;
        bookName = name;
        author = auth;
        price = pr;
    }
    public int getBookId() {
        return bookId;
    }
    public void setBookId(int id) {
        bookId = id;
    }
    public String getBookName() {
        return bookName;
    }
    public void setBookName(String name) {
        bookName = name;
    }
    public String getAuthor() {
        return author;
    }
    public void setAuthor(String auth) {
        author = auth;
    }
    public double getPrice() {
        return price;
    }
    public void setPrice(double pr) {
        price = pr;
    }
    public void display() {
        System.out.println("Book ID   : " + bookId);
        System.out.println("Book Name : " + bookName);
        System.out.println("Author   : " + author);
        System.out.println("Price    : " + price);
    }
}
// Part C: Inheritance
class Person {
    String name;
    int age;
    void display() {
        System.out.println("Name : " + name);
        System.out.println("Age  : " + age);
    }
}
class Student extends Person {
    String course;
    void display() {
        super.display();
        System.out.println("Course : " + course);
    }
}
class Faculty extends Person {
    String department;
    void display() {
        super.display();
        System.out.println("Department : " + department);
    }
}
// Part D: Method Overloading
class Area {
    void area(double radius) {
        System.out.println("Circle Area = " + (3.14 * radius * radius));
    }
    void area(int length, int breadth) {
        System.out.println("Rectangle Area = " + (length * breadth));
    }
}
// Part D: Method Overriding
class Vehicle {
    void display() {
        System.out.println("This is a vehicle.");
    }
}
class Car extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a car.");
    }
}
class Bike extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a bike.");
    }
}
```

```

}
// Part E: Abstraction
abstract class Shape {
    abstract void draw();
}
class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Circle");
    }
}
class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}
// Part E: Interface
interface Printable {
    void print();
}
class Report implements Printable {
    @Override
    public void print() {
        System.out.println("Printing Report...");
    }
}
}
public class Library {
    public static void main(String[] args) {
        System.out.println(" BOOK DETAILS");
        Book b = new Book(101,"Java Programming","James Gosling",500);
        b.display();
        System.out.println("\n INHERITANCE");
        Student s = new Student();
        s.name = "Bob";
        s.age = 25;
        s.course = "Engineering";
        s.display();
        System.out.println("\n faculty");
        Faculty f = new Faculty();
        f.name = "Dr. Bob";
        f.age = 45;
        f.department = "Engineering";
        f.display();
        System.out.println("\n METHOD OVERLOADING");
        Area a = new Area();
        a.area(5.0);
        a.area(10, 20);
        System.out.println("\n METHOD OVERRIDING");
        Vehicle v1 = new Car();
        Vehicle v2 = new Bike();
        v1.display();
        v2.display();
        System.out.println("\n ABSTRACTION ");
        Shape sh1 = new Circle();
        Shape sh2 = new Rectangle();
        sh1.draw();
        sh2.draw();
        System.out.println("\n INTERFACE ");
        Printable p = new Report();
        p.print();
    }
}

```

6. OUTPUT / EXECUTION

BOOK DETAILS

Book ID : 101
Book Name : Java Programming
Author : James Gosling
Price : 500.0

INHERITANCE

Name : Bob
Age : 25
Course : Engineering

faculty

Name : Dr. Bob
Age : 45
Department : Engineering

METHOD OVERLOADING

Circle Area = 78.5
Rectangle Area = 200

METHOD OVERRIDING

This is a car.
This is a bike.

ABSTRACTION

Drawing a Circle
Drawing a Rectangle

INTERFACE

Printing Report...

7. OUTPUT EXPLANATION

- Book Details: The Book object displays its ID, name, author and price using the display() method.
- Inheritance: Student and Faculty inherit name and age from Person and add their own properties.
- Method Overloading: area(5.0) calculates the circle area as 78.5 and area(10, 20) calculates the rectangle area as 200.
- Method Overriding: Vehicle references containing Car and Bike objects execute their overridden display() methods.
- Abstraction: Circle and Rectangle provide their own implementations of the abstract draw() method.
- Interface: Report implements Printable and executes the print() method.

8. RESULT

The Library program was successfully implemented and executed. The program demonstrates encapsulation, inheritance, compile-time polymorphism through method overloading, runtime polymorphism through method overriding, abstraction, and interfaces.

9. LEARNING OUTCOME

Understood the fundamental OOP concepts in Java and learned how classes, objects, constructors, access control, inheritance, polymorphism, abstract classes and interfaces are used to build modular and reusable programs.